Pose Blender Lite - Documentation

Ver. 2.1.0
Made by: ChiefValentine, BSS

What Does It Do?

The **Pose Blender Lite** suite offers a range of tools designed to enhance character animation and interaction in your Unity projects. It enables more natural transitions and greater control over your character's pose by providing the following key features:

- **Pose Blender Lite:**
  Seamlessly blend between animations by overriding specific bone transforms at runtime. This component lets you sample an animation frame and overlay that frame onto selected bones, creating smooth and natural transitions. You can also manually adjust rotation offsets and blend weights for precise control over the final pose.

- **IK Controller:**
  Address the limitations of traditional animation by applying inverse kinematics (IK). This controller automatically adjusts hand or arm positions to match your character's movements, ensuring that interactions with the environment appear realistic.

- **PoseRecorder:**
  Easily capture and store any desired pose or animation. With a single click, record the current state and generate a new AnimationData ScriptableObject (AnimationDataSO), making it simple to reuse or modify poses as needed.

- **Camera Stabilizer:**
  Enhance camera movement by synchronizing it with your character's animations. This tool stabilizes the camera by considering the head and spine transforms, ensuring smooth and responsive visual tracking throughout the scene.

How It Works:
Pose Blender Lite applies its updates during the Late Update phase, ensuring that all modifications—from pose blending to IK corrections—are applied after the base animations are processed. This timing guarantees that your adjustments are smoothly integrated into the final output.

Getting Started

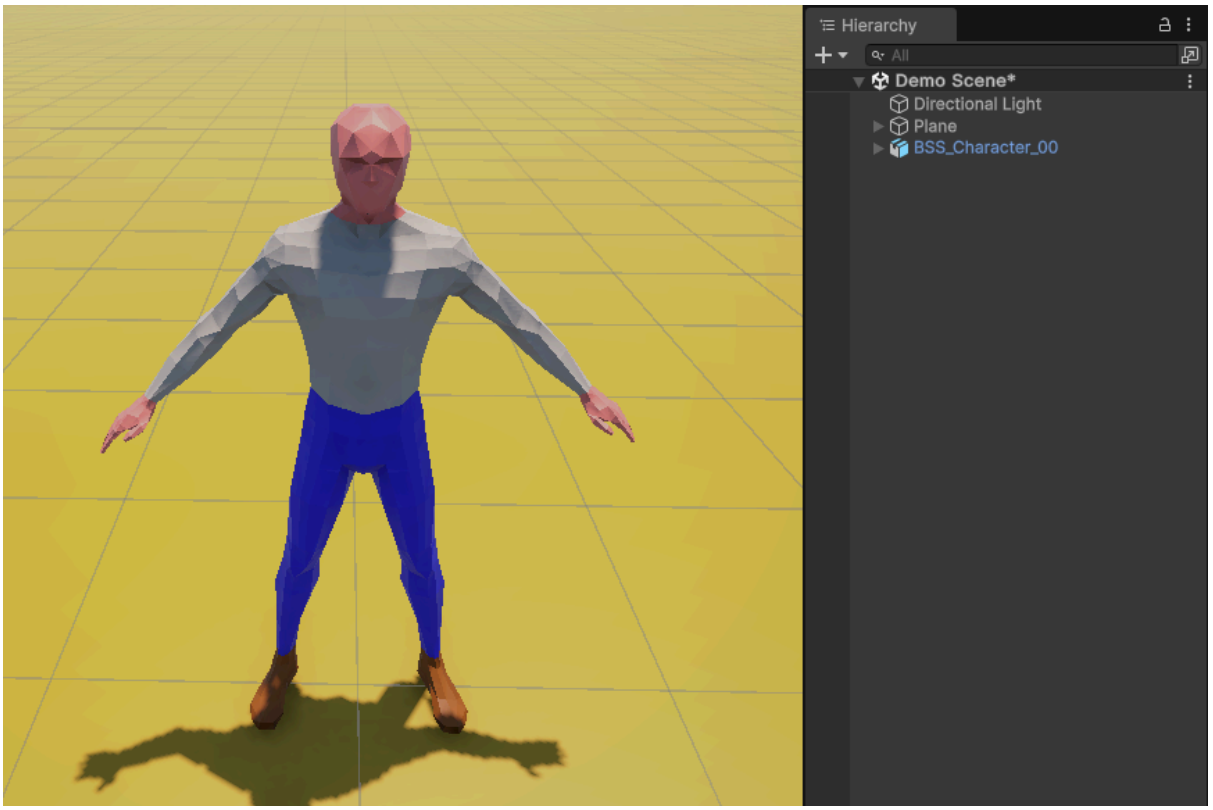
Setting up Pose Blender Lite is simple and can be broken down into a few easy steps:

Step 0: Prepare Your Script Execution Order

- Go to **Edit - Project Settings - Script Execution Order**
- Set the Execution order to be as followed:
    - First PoseBlenderLite
    - Stabilizer Lite
    - IKController

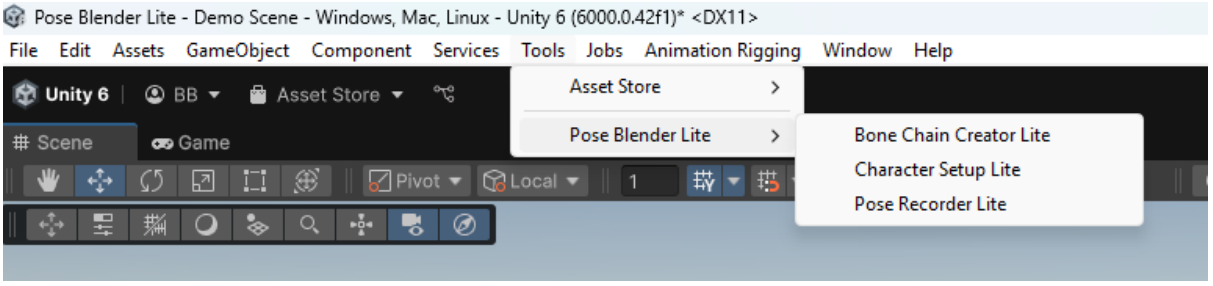
Step 1: Prepare Your Character

- **Drag Your Character into the Scene:**
  Ensure your character has an Animation Controller with at least one animation. To set up the Simple Character Controller that comes with Pose Blender. Add the PBL_Simple_Animation_Controller to your Character.



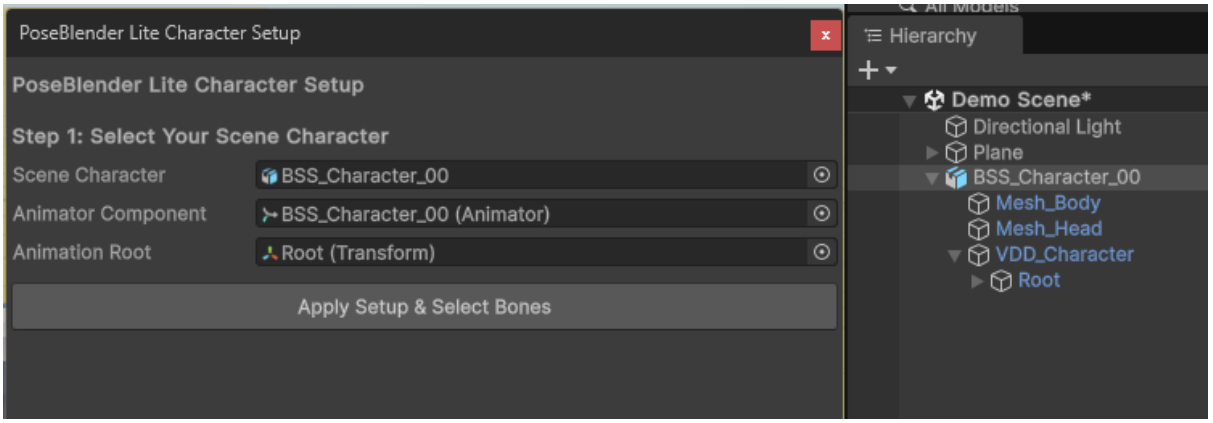
Step 2: Add and Configure Pose Blender Lite

2a. Go to -> Tools > Pose Blender Lite -> Character Setup Lite

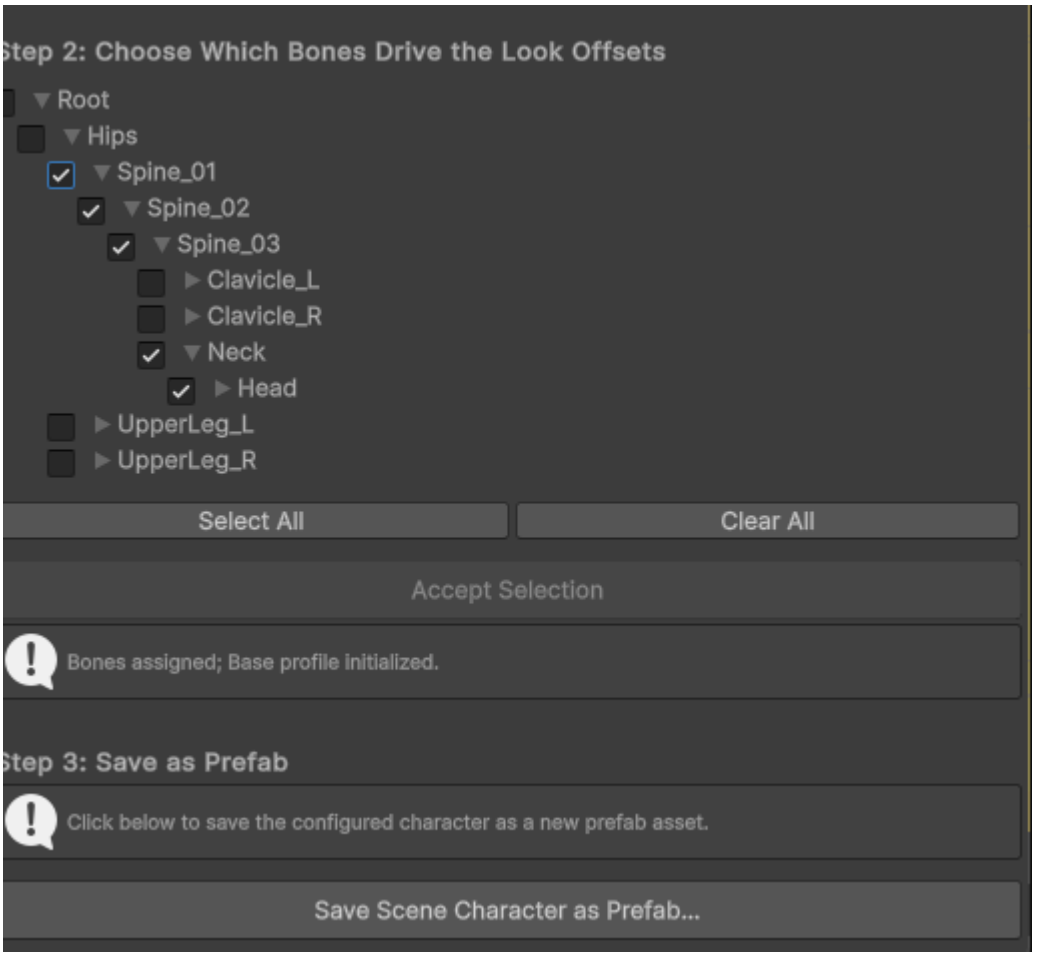


Complete the setup.

Add you Character
Add the animator of your character
Add the animation root of your Character, usually the parent of the Hips bone.

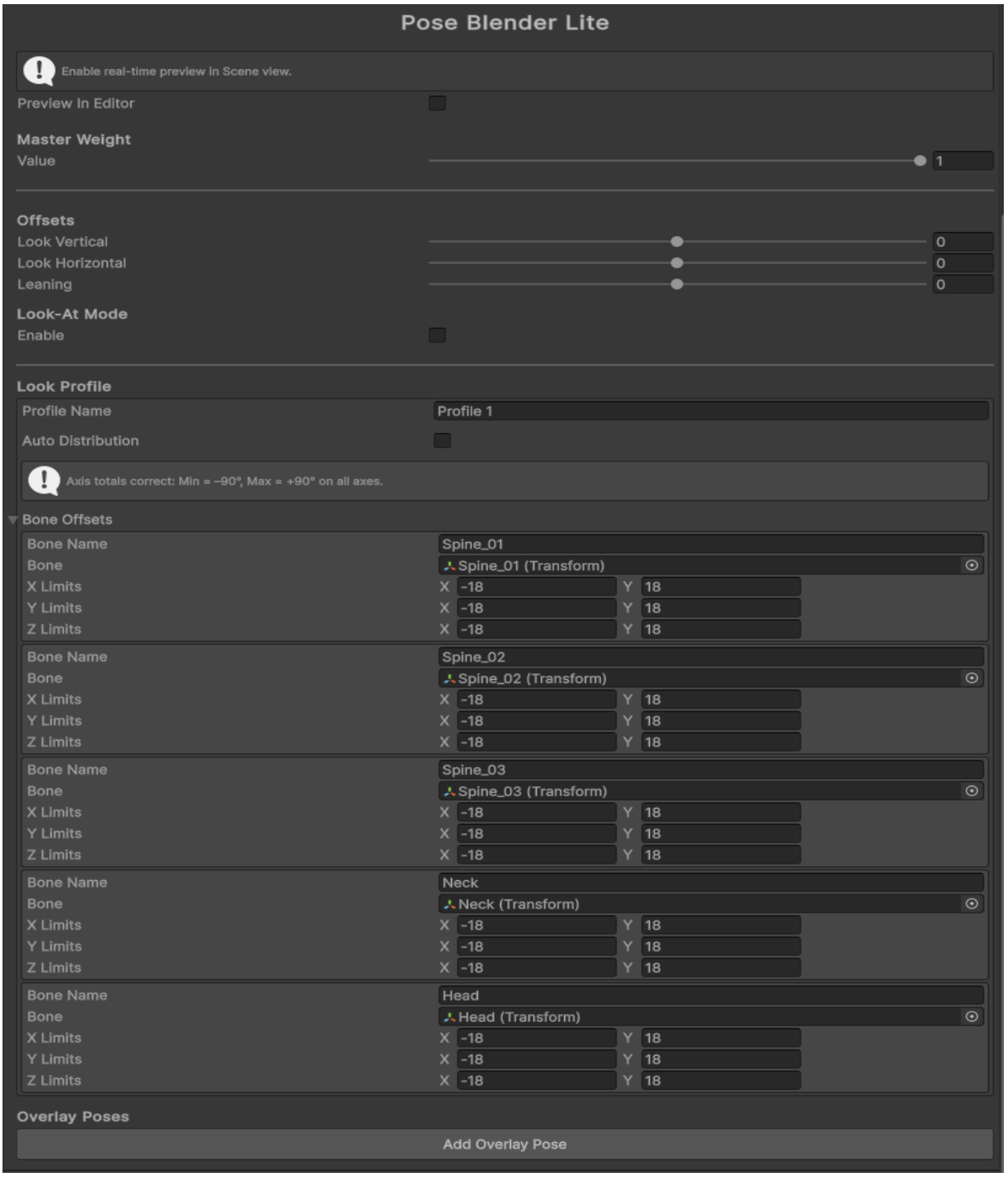


2b. Configure the look offset bones. These bones drive Pose Blender Lite. We choose the full spine, neck and head.



2c. Hitting Save Scene Character as prefab will prompt you with a window to save the Character.

2d. Pose Blender Lite will now be added to your Character. The base offsets have been calculated based on you chosen bones during the setup.



- **Feedback:**
  When the offset bones are configured correctly, you will see a confirmation message in the inspector.

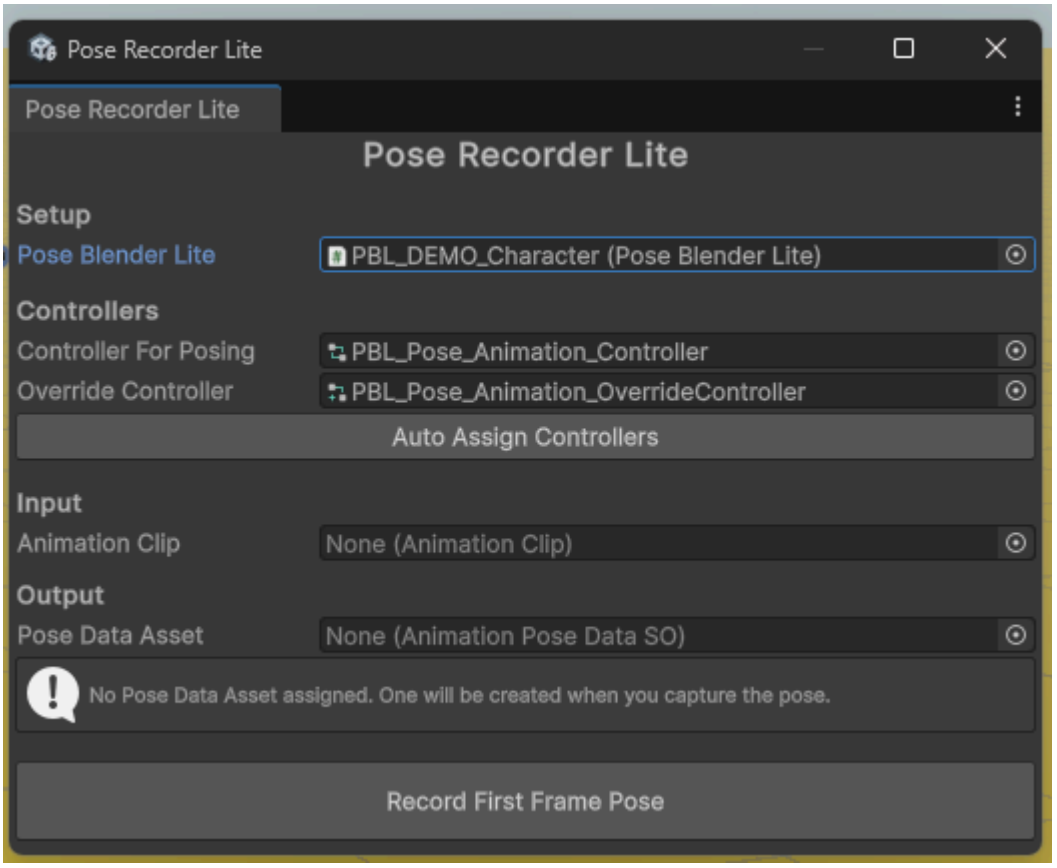
- **Preview in Editor:**
  Enable the preview mode to see real-time adjustments using the character offsets.

Step 3: Creating Animation Data

Before you can set up an overlay pose, you need to capture the animation data that represents the desired pose.

3a. Go to -> Tools > Pose Blender Lite > Pose Recorder lite

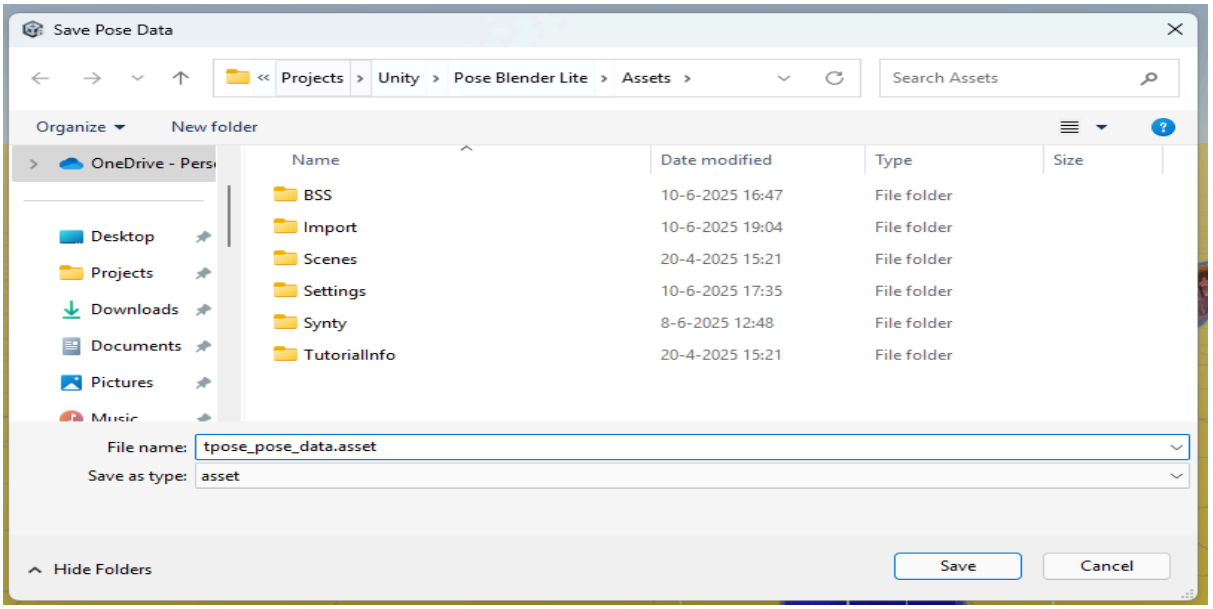
- Ensure PoseRecorder setup matches Pose Blender Lite (assigned Animator and Animation Root).
- Verify **POSE_CONTROLLER** and **POSE_OVERRIDE_CONTROLLER** assignments.
- You are now ready to record Pose Data



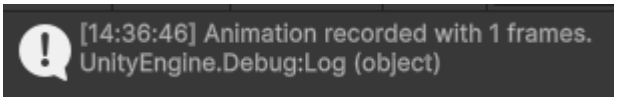
3b. Record a Pose

- Select the animation clip you want to sample and drag it in the Animation Clip Reference. *It will only record the pose of the first frame of the animation.*
    - *Make sure this animation is compatible with your current character.*
- If no Pose Data Asset is referenced, a new one will be created.
- Click 'Record First Frame Pose' to create a new Pose Data Asset.
    - A window will pop up asking you where you want to save the Pose Data Asset.
    - Select save location for new Pose Data asset.

If a Pose Data Asset has been referenced the Pose Data will be overwritten.



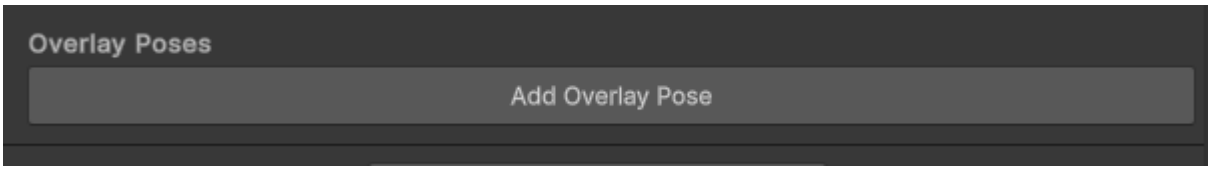
- Console confirms the number of frames captured; the overlay pose uses only the first frame.



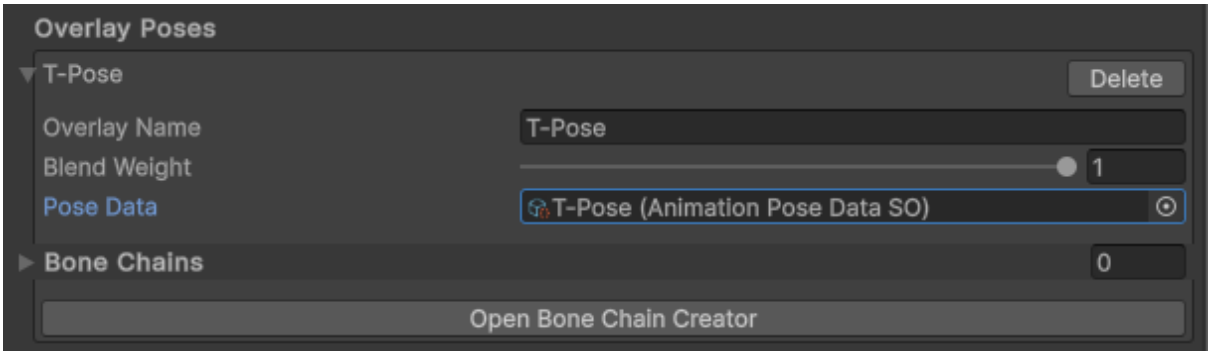
Step 4: Setting Up the Pose Overlay

With your Animation Data ready, you can now apply it as an overlay pose to blend with your character's base animation.

1. **Add an Overlay Pose:**

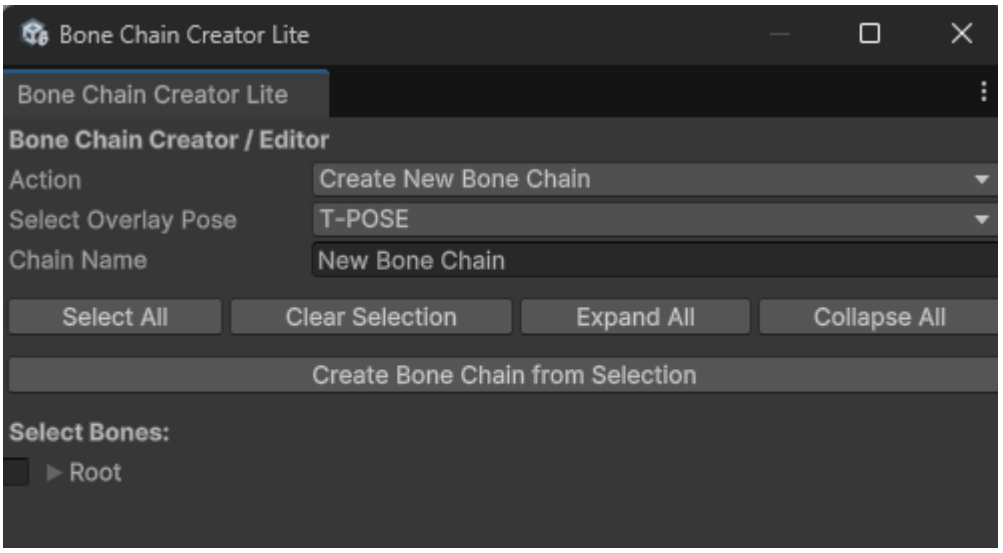


   - Add a new overlay pose.
   - Assign a descriptive name to the overlay and link the newly created Pose Data asset.

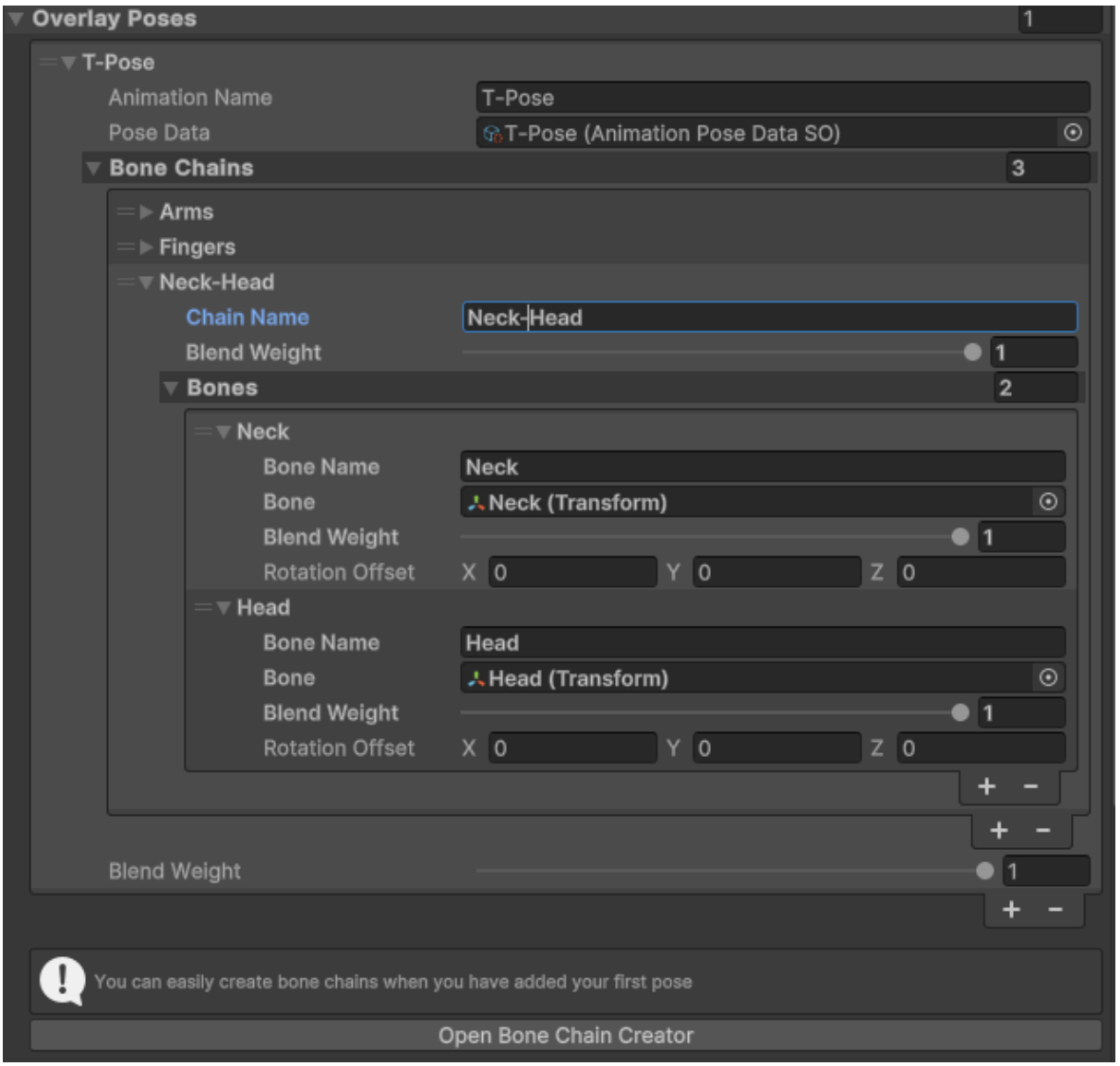


2. **Configure Bone Chains:**

    - Click **Open Bone Chain Creator** to launch the Bone Chain Wizard.



- Use the wizard to create and edit bone chains
    - Choose the action you would like to do
    - Select the pose to apply it to.
    - Give the chain a name.
    - Select the bones you want the chain to have.
    - Hit ‘Create Bone Chain from Selection’
    - Your bone chain will now be added to your overlay pose.



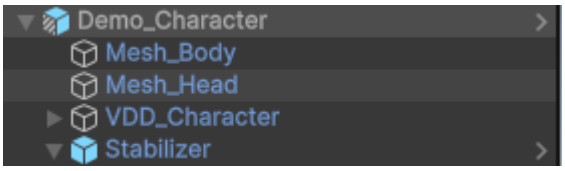
- If the blend weight is set to 1, all bones in that chain will fully rotate toward their Pose Data rotation.

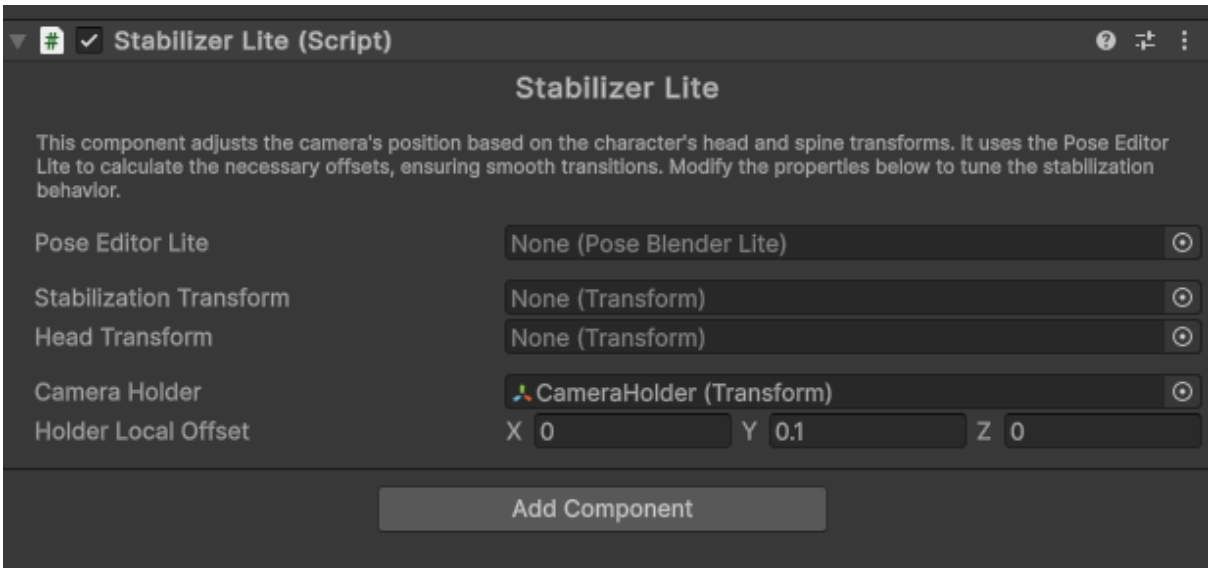
- Adjust individual blend weights as needed for more natural blends between the current animation and the pose.

- If the overlay isn't working immediately, it's likely because the bone chains that the pose affects haven't been assigned.

Step 5: Setting Up Stabilizer Lite

Stabilizer Lite is used to stabilize your camera. Add the Camera Stabilizer to your character, make sure your character is using Pose Blender Lite.





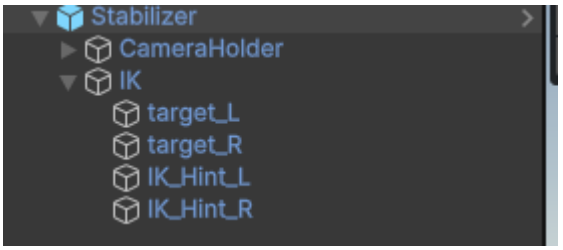
Configuring the Camera Stabilizer:

- Ensure **Stabilizer Lite** references your character's **Pose Blender Lite**.
- Assign the head transform to position the **CameraHolder** accurately.
- Assign the spine transform to the uppermost spine connected to the arms for optimal stabilization results.
- **CameraHolder** should be a child of the stabilizer. Attached to the **CameraHolder** transform is the **Camera** object.
- **Holder Local Offset** lets you offset the camera holder local position.

Step 6: Setting Up IK

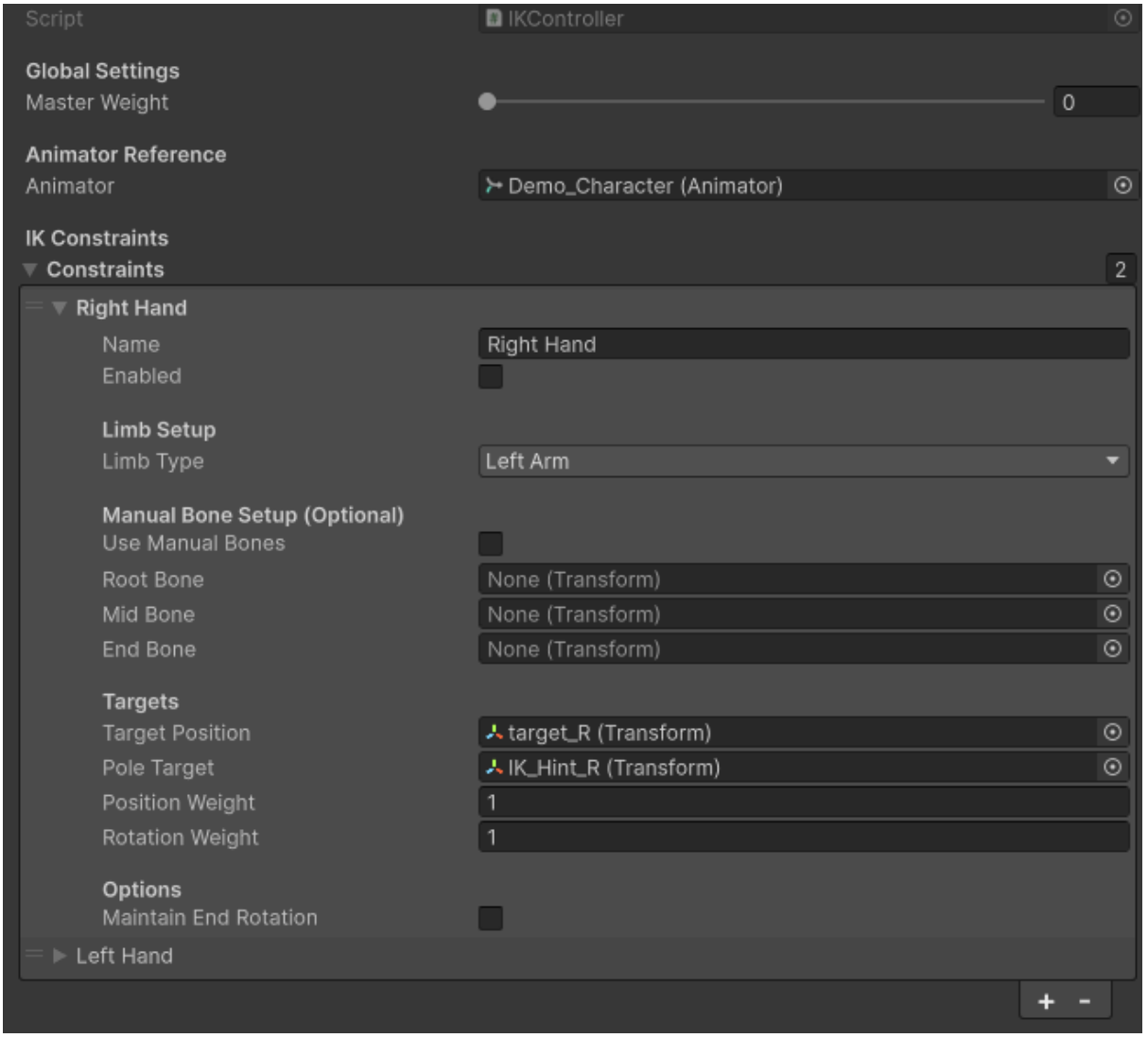
When Pose Blender Lite is added, the IK Controller is automatically included. Ensure proper execution order by adding both scripts to the Unity script execution order. The IK Controller must run after Pose Blender Lite.

- ***IK SETUP INDICATIVE ONLY -***



Configuring IK:

- Add IK constraints to the provided list.
- Name the limb and select its type.
- For Humanoid characters, manual bone setup is optional but available.
- Assign IK targets and hints appropriately. The demo character demonstrates this by synchronizing IK targets and hints with Pose Blender Lite rotations, ensuring natural and coherent movements.



PoseBlenderLite – Public API Reference

PoseBlenderLite Methods

SetLookOffsets(float vertical, float horizontal, float leaning)

Sets the look offsets for vertical, horizontal, and leaning angles.
All values are clamped to **–90° to +90°**.

Usage Example:

```
poseBlenderLite.SetLookOffsets(30f, 15f, 5f);
```

SetVerticalOffset(float offset)

Directly sets the vertical (up/down) look offset.
Value is clamped to –90° to +90°.

Usage Example:

```
poseBlenderLite.SetVerticalOffset(25f);
```

SetHorizontalOffset(float offset)

Directly sets the horizontal (left/right) look offset.
Value is clamped to –90° to +90°.

Usage Example:

```
poseBlenderLite.SetHorizontalOffset(-15f);
```

SetLeaningOffset(float offset)

Directly sets the leaning angle.
Value is clamped to –90° to +90°.

Usage Example:

```
poseBlenderLite.SetLeaningOffset(10f);
```

BlendOverlay(string overlayName, float targetBlend, float duration)

Smoothly blends a **single** overlay pose to the target blend weight over the specified duration.

Parameters:

- **overlayName** – Name of the overlay animation
- **targetBlend** – Target weight (0–1)
- **duration** – Transition time in seconds

Usage Example:

```
poseBlenderLite.BlendOverlay("Aiming", 1f, 0.3f);
```

BlendAllOverlays(float targetBlend, float duration)

Smoothly blends **all** overlay poses to the same blend weight.

Parameters:

- **targetBlend** – Target weight for all overlays
- **duration** – Transition time in seconds

Usage Example:

```
poseBlenderLite.BlendAllOverlays(0f, 0.5f);
```

InitializeBaseLookProfile()

Creates the first look profile (“Profile 1”) by distributing rotation ranges across all selected look bones.
Called during initial setup.

Usage Example:

```
poseBlenderLite.InitializeBaseLookProfile();
```

RebuildCurrentLookConfig()

Rebuilds the runtime active look configuration from profile data.
Useful after editing profiles or changing the active profile.

Usage Example:

```
poseBlenderLite.RebuildCurrentLookConfig();
```

AddLookProfile(string newName = null)

Creates a new look profile using the configured look bones.
If no name is supplied, assigns a default (e.g., “Profile 2”).

Parameters:

- **newName** – Optional custom profile name

Usage Example:

```
poseBlenderLite.AddLookProfile("Combat Stance");
```

BlendToProfile(int targetIndex, float duration)

Smoothly transitions from the current profile to the target profile.
Blends all bone rotation limits over the specified duration.

Parameters:

- **targetIndex** – Index inside the `lookConfigs` list
- **duration** – Blend time in seconds

Usage Example:

```
poseBlenderLite.BlendToProfile(1, 0.5f);
```

IKController Methods

SetMasterWeight(float weight)

Sets the global IK weight (0–1).
Affects all IK constraints.

Usage Example:

```
ikController.SetMasterWeight(0.75f);
```

SetLayerWeightByName(string layerName, float blendWeight)

Sets both position and rotation weights for the IK layer with the given name.

Parameters:

- **layerName** – IK layer name
- **blendWeight** – Position/rotation influence (0–1)

Usage Example:

```
ikController.SetLayerWeightByName("Left Hand", 0.8f);
```

SetLayerWeightByName(string layerName, float positionBlendWeight, float rotationBlendWeight)

Sets **position** and **rotation** weights independently for the named IK layer.

Parameters:

- **layerName** – IK layer name
- **positionBlendWeight** – Position weight (0–1)
- **rotationBlendWeight** – Rotation weight (0–1)

Usage Example:

```
ikController.SetLayerWeightByName("Right Hand", 0.8f, 0.6f);
```

BlendMasterWeight(float targetWeight, float durationInSeconds)

Smoothly blends the master IK weight to a new value over time.

Parameters:

- **targetWeight** – Target global IK weight (0–1)
- **durationInSeconds** – Transition time in seconds

Usage Example:

```
ikController.BlendMasterWeight(1f, 0.3f);
```